



Bônus — Projeto com duas tabelas relacionadas + tratamento de erros

Jornada DEV START · Material bônus de autoestudo · Programa START — TOTVS

Paulista Sem entrega, sem nota, sem prazo. É pra você estudar no seu ritmo, com suporte no Discord em [#duvidas-parte2-protheus](#) até **07/08**. **Precisa do ambiente Protheus** (SmartClient + AppServer + Configurator/DevStudio) para executar as telas. Mesmo sem ambiente, você pode ler, entender a lógica e escrever o código.

Antes de começar: este é um projeto do zero, independente do que foi feito em aula

Este bônus é um **projeto de treinamento novo**, criado especialmente para quem não chegou a essa parte do curso ao vivo. Ele **não usa** as tabelas que a turma já criou em aula ([ZA1](#) , por exemplo) — pra não ter risco de colidir com o que você já montou no seu ambiente.

Por isso as tabelas aqui têm nomes diferentes do material original do Módulo 9:

Material do Módulo 9 (dado em aula)	Este bônus (novo, isolado)
SZ1 — Contatos	SZ2 — Contatos
SZ2 — Interações	SZ3 — Interações
Campos Z1_*	Campos Z2_*
Campos Z2_*	Campos Z3_*

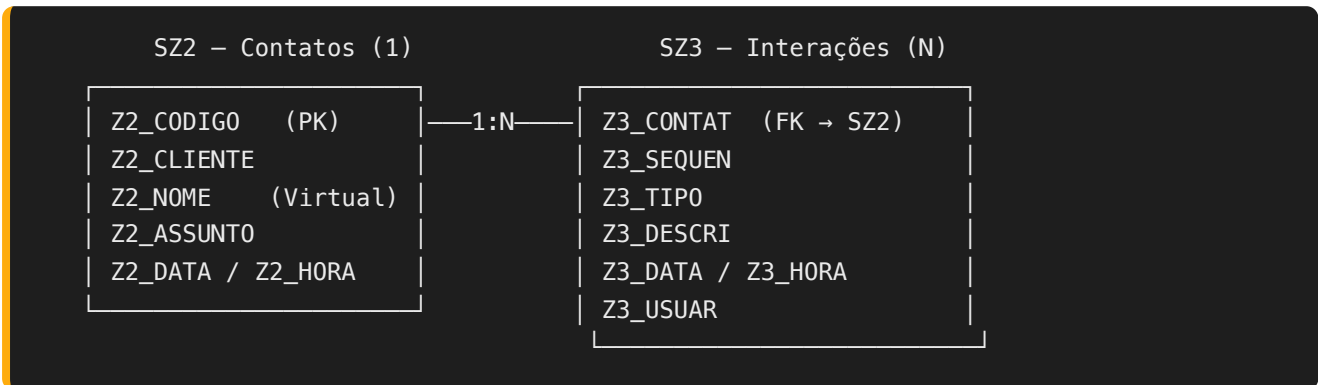
Se você já fez o projeto do Módulo 9 com [SZ1](#) / [SZ2](#) em aula, **ótimo** — os conceitos abaixo são os mesmos, só que aplicados a um par de tabelas novo. Pode fazer este bônus como uma segunda volta, do zero, pra fixar. Se você **não** chegou a fazer, este é o seu primeiro contato completo com o assunto — comece por aqui, sem pressa.

1. O projeto: um mini-sistema de verdade

Você vai construir um sistema de **Controle de Contatos e Histórico de Interações**:

- **SZ2 — Contatos:** registra os contatos de clientes (cliente, assunto, data)
- **SZ3 — Interações:** detalha cada interação feita com aquele contato (ligação, e-mail, visita...)

Cada contato (SZ2) pode ter **várias** interações (SZ3). É uma relação de **um para muitos (1:N)** — a mesma estrutura de pedido→itens, cliente→endereços, nota→parcelas. Aprender a construir **um** relacionamento 1:N ensina você a construir **todos**.



💡 **Regra de ouro do relacionamento:** a tabela “filha” (SZ3) sempre guarda a **chave da mãe** (o campo **Z3_CONTAT** aponta para **Z2_CODIGO**). É por esse campo que ligamos as duas.

2. Modelando os dados (dicionário)

Estrutura da SZ2 — Contatos

Título	Campo	Tipo	Tam	Dec	Contexto
Filial	Z2_FILIAL	C	2	0	Real
Código	Z2_CODIGO	C	6	0	Real
Cliente	Z2_CLIENTE	C	6	0	Real
Loja Cliente	Z2_LOJACLI	C	2	0	Real
Nome Cliente	Z2_NOME	C	40	0	Virtual
Assunto	Z2_ASSUNTO	C	60	0	Real
Data	Z2_DATA	D	8	0	Real
Hora	Z2_HORA	C	5	0	Real

Estrutura da SZ3 – Interações

Título	Campo	Tipo	Tam	Dec	Contexto
Filial	Z3_FILIAL	C	2	0	Real
Contato (→SZ2)	Z3_CONTAT	C	6	0	Real
Seq.	Z3_SEQUEN	C	3	0	Real
Tipo	Z3_TIPO	C	1	0	Real
Descrição	Z3_DESCRI	C	100	0	Real
Data	Z3_DATA	D	8	0	Real
Hora	Z3_HORA	C	5	0	Real
Usuário	Z3_USUAR	C	20	0	Real
Cód. Contato	Z3_CODIGO	C	6	0	Virtual
Assunto Cont.	Z3_ASSUNT	C	60	0	Virtual

Registrar as tabelas (SX2) e os índices (SIX)

SX2 – tabelas:

Prefixo	Nome	Modo
SZ2	Contatos	C (Compartilhado)
SZ3	Interações	C (Compartilhado)

SIX – índices da SZ2:

Ordem	Expressão	Descrição
1	Z2_FILIAL + Z2_CODIGO	Chave primária
2	Z2_FILIAL + Z2_CLIENTE + Z2_LOJACLI	Por cliente
3	Z2_FILIAL + DTOS(Z2_DATA)	Por data

SIX – índices da SZ3:

Ordem	Expressão	Descrição
1	Z3_FILIAL + Z3_CONTAT + Z3_SEQUEN	Chave primária
2	Z3_FILIAL + DTOS(Z3_DATA)	Por data

Domínio dos tipos de interação (SX5)

O campo **Z3_TIP0** guarda **um código de uma letra**. Cadastre uma tabela de domínio no SX5 com código **Z3** :

Chave	Descrição
E	E-mail
L	Ligação
R	Reunião
V	Visita
W	WhatsApp

Campos virtuais

Um **campo virtual** não é gravado no banco — ele é **calculado na hora** de mostrar. Configure no **X3_RELACAO** da SZ3:

Z3_CODIGO (código do contato SZ2):

```
POSICIONE("SZ2", 1, xFilial("SZ2") + M->Z3_CONTAT, "Z2_CODIGO")
```

Z3_ASSUNT (assunto do contato):

```
POSICIONE("SZ2", 1, xFilial("SZ2") + M->Z3_CONTAT, "Z2_ASSUNTO")
```

💡 **POSICIONE(alias, ordem, chave, campo)** “pula” até o registro cuja chave bate e devolve o valor de um campo. É a função que você mais vai usar para buscar dados de outra tabela.

3. A biblioteca de funções comuns (não repita código!)

Antes de escrever as rotinas, centralize as funções que se repetem numa **biblioteca**. Assim, se a regra mudar, você altera **em um só lugar** — o princípio **DRY** (*Don't Repeat Yourself*).

```

// =====
// STTIBLIB.PRW
// Biblioteca de funções comuns do módulo TIB (bônus)
// =====

#include "protheus.ch"

// -----
// NomeCliente
// Retorna o nome do cliente a partir do código/loja
// -----
STATIC FUNCTION NomeCliente(cCodigo, cLoja)
    LOCAL cFilial := xFilial("SA1")
    LOCAL cNome

    IF Empty(cCodigo)
        RETURN ""
    ENDIF

    cNome := POSICIONE("SA1", 1, cFilial + cCodigo + cLoja, "A1_NOME")
RETURN AllTrim(cNome)

// -----
// ProxCodigoSZ2
// Retorna o próximo código disponível para SZ2
// -----
STATIC FUNCTION ProxCodigoSZ2()
RETURN GetSXENum("SZ2", "Z2_CODIGO")

// -----
// ProxSequenSZ3
// Retorna a próxima sequência para um contato na SZ3
// -----
STATIC FUNCTION ProxSequenSZ3(cContat)
    LOCAL nMax := 0

    dbSelectArea("SZ3")
    dbSetOrder(1)
    dbSeek(xFilial("SZ3") + cContat)

    WHILE !Eof() .AND. SZ3->Z3_FILIAL == xFilial("SZ3") .AND. SZ3->Z3_CONTAT == cContat
        nMax := Max(nMax, Val(SZ3->Z3_SEQUEN))
        dbSkip()
    ENDDO

RETURN StrZero(nMax + 1, 3)

// -----
// DescTipoInteracao
// Retorna a descrição do tipo de interação
// -----

```

```
STATIC FUNCTION DescTipoInteracao(cTipo)
    LOCAL cDesc := ""
    DO CASE
        CASE cTipo == "E" ; cDesc := "E-mail"
        CASE cTipo == "L" ; cDesc := "Ligação"
        CASE cTipo == "R" ; cDesc := "Reunião"
        CASE cTipo == "V" ; cDesc := "Visita"
        CASE cTipo == "W" ; cDesc := "WhatsApp"
    ENDCASE
    RETURN cDesc
```

4. A rotina de Contatos (SZ2) — a tela “mãe”

Além do CRUD padrão, ela traz **legendas coloridas** (o usuário vê o status pela cor) e um **botão de navegação** que abre as interações **daquele** contato.

```

// =====
// STTIB003.PRW
// Cadastro de Contatos (mBrowse)
// =====

#include "protheus.ch"

USER FUNCTION STTIB003()

    LOCAL cFiltro := ""
    LOCAL aColors

    PRIVATE cCadastro := "Contatos"
    PRIVATE aRotina := {;
        {"Pesquisar", "AxPesqui", 0, 1},;
        {"Visualizar", "AxVisual", 0, 2},;
        {"Incluir", "AxInclui", 0, 3},;
        {"Alterar", "AxAltera", 0, 4},;
        {"Excluir", "AxDeleta", 0, 5},;
        {"Interações", "U_STTIB004", 0, 6}; // ← abre a SZ3 filtrada!
    }

    // Legenda por idade do contato:
    // Verde: recente (até 7 dias)
    // Amarelo: antigo (8 a 30 dias)
    // Vermelho: muito antigo (> 30 dias)
    aColors := {;
        {"SZ2->Z2_DATA >= dDataBase - 7", "BR_GREEN"},;
        {"SZ2->Z2_DATA >= dDataBase - 30", "BR_YELLOW"},;
        {"T.", "BR_RED"};
    }

    dbSelectArea("SZ2")
    dbSetOrder(1)
    dbSeek(xFilial("SZ2"))

    mBrowse(1, 1, 22, 75, "SZ2", , , , , aColors, , , , .F., , , cFiltro)

    RETURN NIL

```


Elemento	O que faz
<code>aRotina</code>	Lista de opções do menu do browse. Cada item: <code>{"Título", "Função", 0, Operação}</code>
<code>{"Interações", "U_STTIB004", 0, 6}</code>	Opção customizada — chama outra rotina (nossa!)
<code>aColors</code>	Regras de cor: a primeira que der <code>.T.</code> pinta a linha
<code>mBrowse(...)</code>	Monta a grade (browse) com todos os recursos configurados

5. A rotina de Interações (SZ3) — a tela “filha”

Quando o usuário clica em “**Interações**” na tela de Contatos, o Protheus chama a rotina abaixo. O segredo está em **capturar o contato atual** e **filtrar** a SZ3 só por ele:

```
// =====
// STTIB004.PRW
// Interações de um Contato (SZ3)
// Chamada a partir da STTIB003 (botão "Interações")
// =====

#include "protheus.ch"

USER FUNCTION STTIB004()

    LOCAL cFiltro    := ""
    LOCAL cCodigSZ2 := SZ2->Z2_CODIGO    // código do registro atual da SZ2

    PRIVATE cCadastro := "Interações - Contato " + AllTrim(cCodigSZ2)
    PRIVATE aRotina    := {;
        {"Pesquisar", "AxPesqui", 0, 1},;
        {"Visualizar", "AxVisual", 0, 2},;
        {"Incluir", "AxInclui", 0, 3},;
        {"Alterar", "AxAltera", 0, 4},;
        {"Excluir", "AxDeleta", 0, 5};
    }

    // Filtro: mostrar apenas interações deste contato
    cFiltro := "Z3_CONTAT == '" + cCodigSZ2 + "'"

    dbSelectArea("SZ3")
    dbSetOrder(1)
    dbSeek(xFilial("SZ3") + cCodigSZ2)

    mBrowse(1, 1, 22, 75, "SZ3", , , , , , , , , , .F., , , cFiltro)

    RETURN NIL
```

Gatilhos: o sistema preenche por você

Configure gatilhos (SX7) na SZ3 para preencher data, hora e usuário automaticamente:

Campo	Regra do gatilho	Fase
Z3_DATA	dDataBase	1 (só na inclusão)
Z3_HORA	IF(INCLUI, Time(), SZ3->Z3_HORA)	3 (ambos)
Z3_USUAR	cNomUsr (usuário logado)	1 (só na inclusão)

💡 `dDataBase` é a data-base do sistema; `Time()` é a hora atual; `cNomUsr` é o nome do usuário logado. Com esses três gatilhos, o usuário só digita o que interessa: **tipo e descrição**.

6. Validações cruzadas e integridade referencial

Duas tabelas relacionadas trazem uma responsabilidade nova: garantir que a ligação entre elas faça sentido. Isso se chama **integridade referencial** — não pode existir “filho órfão” nem apagar uma “mãe” que ainda tem filhos.

1. Toda interação precisa apontar para um contato que existe

No `X3_VALID` do campo `Z3_CONTAT`:

```
ExistCpo("SZ2", xFilial("SZ2") + M->Z3_CONTAT, 1)
```

`ExistCpo` verifica se a chave existe na tabela. Se não existir, o Protheus barra a digitação e avisa o usuário — impossível cadastrar interação para um contato inexistente.

2. Não deixe apagar um contato que tem interações

Se o usuário apagar um contato (SZ2) que ainda tem interações (SZ3), essas interações viram “órfãs”. Para impedir, crie uma validação de exclusão:

```
// =====  
// Impede excluir Contato (SZ2) com Interações (SZ3)  
// =====  
USER FUNCTION VALEXCSZ2()  
  
    IF ExistCpo("SZ3", xFilial("SZ3") + SZ2->Z2_CODIGO, 1)  
        MsgAlert("Contato possui interações! Exclua as interações primeiro.", "Atenção")  
        RETURN .F.    // cancela a exclusão  
    ENDIF  
  
RETURN .T.    // libera a exclusão
```

⚠ **Validação de dados ≠ tratamento de erro.** Aqui você está validando uma **regra de negócio** (“não pode apagar mãe com filhos”) — para isso, `IF` + `RETURN .F.` basta. Guarde essa distinção: ela é a ponte para a próxima seção.

7. “E quando algo dá errado?” — Tratamento de erros

Você já validou o que o **usuário** digita. Mas e quando o problema **não é** o usuário?

Imagine a rotina de salvar um contato. Você validou tudo, chamou o comando de gravação... e nesse exato instante **o banco de dados fica inacessível**, a **rede cai** ou **outro usuário travou o registro**. Sem tratamento, o Protheus mostra uma mensagem técnica assustadora, os dados ficam **pela metade**, e o usuário liga furioso. Isso **não pode** acontecer num sistema profissional.

Sem tratamento	Com tratamento
Trava com mensagem técnica	Mensagem clara e amigável
Dados em estado inconsistente	Rollback automático da operação
Difícil de diagnosticar	Log técnico detalhado
Usuário perdido	Sistema continua funcionando

Os tipos de erro (e como tratar cada um)

Tipo	Quando ocorre	Trata com
Validação	Dados inválidos (CPF, campo vazio)	<code>IF / RETURN .F.</code>
Negócio	Regra da empresa violada (estoque, prazo)	<code>IF / RETURN .F.</code>
Runtime	Divisão por zero, acesso a NIL	<code>BEGIN SEQUENCE</code>
Banco de dados	Lock, conexão perdida	<code>BEGIN SEQUENCE</code>
Arquivo/Rede	Arquivo não encontrado, disco cheio	<code>BEGIN SEQUENCE</code>

Os dois primeiros você resolve com `IF`. Os três últimos são **inesperados** — para eles existe o `BEGIN SEQUENCE`.

BEGIN SEQUENCE — o “try/catch” do ADVPL

```
BEGIN SEQUENCE
    // código que pode gerar erro
    // se ocorrer um erro, a execução SALTA para o RECOVER
RECOVER WITH oErro
    // executado só em caso de erro
    // oErro traz os detalhes do que aconteceu
END SEQUENCE

// o programa continua daqui, com ou sem erro
```

Um exemplo simples, para ver o mecanismo funcionando (este roda até em Harbour puro):

```
FUNCTION Main()
    LOCAL nA := 10
    LOCAL nB := 0
    LOCAL nResultado

    BEGIN SEQUENCE
        nResultado := nA / nB    // divisão por zero!
        QQout("Resultado: " + Str(nResultado))
    RECOVER WITH oErro
        QQout("Erro capturado!")
        QQout("Descrição: " + oErro:Description)
    END SEQUENCE

    QQout("Programa continua normalmente.")
    RETURN NIL
```

Saída:

```
Erro capturado!
Descrição: Zero divisor
Programa continua normalmente.
```

Sem o `BEGIN SEQUENCE`, aquela divisão por zero **derrubaria** o programa inteiro.

O objeto de erro (oErro)

Ao capturar com `RECOVER WITH oErro`, você recebe um objeto cheio de informações úteis:

Propriedade	Descrição
<code>oErro:Description</code>	Descrição do erro
<code>oErro:Operation</code>	Operação que causou o erro
<code>oErro:SubSystem</code>	Subsistema (DBFNTX, NETSIO...)
<code>oErro:SubCode</code>	Código do erro
<code>oErro:ProcName</code>	Função onde o erro ocorreu
<code>oErro:ProcLine</code>	Linha onde o erro ocorreu

8. Juntando tudo: a rotina de salvar profissional

Uma gravação profissional combina **quatro camadas**: validação de negócio (`IF`), **transação** (para poder desfazer), `BEGIN SEQUENCE` (para capturar o inesperado) e **log** (para diagnosticar depois).

```

// =====
// Gravação segura de um Contato (SZ2)
// =====
USER FUNCTION STTIB003SALVAR()

    LOCAL lok := .T.
    LOCAL oErro

    BeginTran()    // 1. inicia a transação (tudo ou nada)

    BEGIN SEQUENCE

        // 2. validação de negócio → IF/Break
        IF Empty(M->Z2_CLIENTE)
            MsgAlert("Cliente é obrigatório!", "Atenção")
            lok := .F.
            Break()    // força a saída para o RECOVER (sem erro real)
        ENDIF

        // 3. operação no banco, protegida por lock
        dbSelectArea("SZ2")
        IF INCLUI
            RecLock("SZ2", .T.)    // .T. = novo registro
        ELSE
            RecLock("SZ2", .F.)    // .F. = altera o atual
        ENDIF

        SZ2->Z2_CODIGO    := M->Z2_CODIGO
        SZ2->Z2_CLIENTE    := M->Z2_CLIENTE
        SZ2->Z2_ASSUNTO    := M->Z2_ASSUNTO
        MsUnLock()        // salva e libera o lock

    RECOVER WITH oErro
        // 4. deu erro inesperado: desfaz tudo, avisa e loga
        lok := .F.
        RollBackTran()
        MsgStop("Erro ao salvar: " + oErro:Description, "Erro")
        U_GRAVARLOGB("STTIB003SALVAR", oErro)
        RETURN lok
    END SEQUENCE

    IF lok
        CommitTran()    // 5. tudo certo: confirma a transação
    ENDIF

    RETURN lok

```

Por que transação (`BeginTran`) e lock (`RecLock`)?

- `BeginTran` / `CommitTran` / `RollBackTran` garantem o princípio “**tudo ou nada**”: se algo falhar no meio, o `RollBackTran()` desfaz **todas** as gravações — nada fica pela metade.
- `RecLock` / `MsUnLock` garantem que **dois usuários não editem o mesmo registro ao mesmo tempo**. `RecLock("SZ2", .T.)` cria um registro novo; `RecLock("SZ2", .F.)` trava o atual para alteração; `MsUnLock()` salva e libera.

A função de log

O log é o que permite **diagnosticar em produção** um erro que você não viu acontecer:

```
// =====  
// STTIBLIB.PRW – adicione esta função à sua lib  
// =====  
USER FUNCTION GRAVARLOGB(cFuncao, oErro)  
  
    LOCAL cArqLog := "\\logs\advpl_" + DToS(Date()) + ".log"  
    LOCAL nHandle  
    LOCAL cLinha  
  
    cLinha := DToS(Date()) + " " + Time() + " | "  
    cLinha += cFuncao + " | "  
    cLinha += cNomUsr + " | "  
    cLinha += "Empresa: " + cEmpAnt + " Filial: " + cFilAnt + " | "  
  
    IF oErro != NIL  
        cLinha += "ERRO: " + oErro:Description + " | "  
        cLinha += "Func: " + oErro:ProcName + ":" + cValToChar(oErro:ProcLine) + " | "  
        cLinha += "SubSist: " + oErro:SubSystem + " | "  
        cLinha += "Oper: " + oErro:Operation  
    ENDIF  
  
    nHandle := FOpen(cArqLog, FO_READWRITE + FO_SHARED)  
    IF nHandle < 0  
        nHandle := FCreate(cArqLog)  
    ENDIF  
    FSeek(nHandle, 0, FS_END) // vai para o fim do arquivo  
    FWrite(nHandle, cLinha + CRLF)  
    FClose(nHandle)  
  
    RETURN NIL
```

ERRORBLOCK: uma rede de segurança global

E se um erro escapar de **todos** os `BEGIN SEQUENCE` ? O `ErrorBlock()` instala um **tratador global** — a última rede de proteção do módulo:


```

USER FUNCTION MEUMENUB()

    LOCAL bOldError := ErrorBlock({|oE| U_MEUTRATADORB(oE)}) // instala

    U_STTIB003() // todo o módulo roda "protegido"

    ErrorBlock(bOldError) // restaura o handler original ao sair

RETURN NIL

USER FUNCTION MEUTRATADORB(oErro)
    MsgStop("Ocorreu um erro inesperado. A equipe de TI foi notificada.", "Erro")
    U_GRAVARLOGB("ERRO GLOBAL", oErro)
RETURN Break(oErro) // propaga o erro após tratar

```

Boas práticas (o resumo que vale ouro)

1. **IF** para o esperado, **BEGIN SEQUENCE** para o inesperado. Validação de dados é **IF** ; erro de banco/rede é **BEGIN SEQUENCE** .
2. Toda operação em transação faz rollback no erro. **RollBackTran()** no **RECOVER** , sempre.
3. Mensagem amigável para o usuário, detalhe técnico no log. O usuário não precisa ver **oErro:SubCode** — o desenvolvedor sim.
4. Nunca engula um erro em silêncio. Um **RECOVER** vazio é uma bomba-relógio: no mínimo, grave o log.

9. O menu e a montagem final

Para o usuário chegar às rotinas, configure o menu no **SIGACFG > Menu do Sistema**:

```

SIGACOM
├─ Cadastros
│   ├── Contatos (bônus)      → USER FUNCTION STTIB003
│   └─ Interações (todas, bônus) → USER FUNCTION STTIB004B

```

E, nas rotinas, aponte a biblioteca para reaproveitar as funções:

```

// no appserver.ini ou via SET PROCEDURE
SET PROCEDURE TO STTIBLIB

```

Estrutura final do projeto:

STTIBLIB.PRW ← biblioteca de funções comuns (+ GRAVARLOGB)
 STTIB003.PRW ← Contatos (SZ2) – mBrowse com legendas + botão Interações
 STTIB004.PRW ← Interações (SZ3) – filtrada pelo contato selecionado
 STTIB004B.PRW ← Interações (SZ3) – listagem geral, para o menu

Referência rápida

```
// --- Navegação entre tabelas ---
POSICIONE("SZ2", 1, xFilial("SZ2") + cChave, "Z2_NOME") // busca campo de outra
tabela
ExistCpo("SZ2", xFilial("SZ2") + cChave, 1)           // .T. se a chave existe

// --- Browse com opção customizada e legendas ---
PRIVATE aRotina := { {"Interações", "U_STTIB004", 0, 6} }
aColors := { {"SZ2->Z2_DATA >= dDataBase - 7", "BR_GREEN"}, {"T.", "BR_RED"} }
mBrowse(1, 1, 22, 75, "SZ2", , , , , aColors, , , , .F., , , cFiltro)

// --- Gravação segura (transação + lock + erro) ---
BeginTran()
BEGIN SEQUENCE
  RecLock("SZ2", .T.) // .T. inclui .F. altera
  SZ2->Z2_CODIGO := M->Z2_CODIGO
  MsUnLock()       // salva e libera o lock
  CommitTran()     // confirma
RECOVER WITH oErro
  RollBackTran()   // desfaz tudo
  MsgStop("Erro: " + oErro:Description, "Erro")
  U_GRAVARLOGB("Salvar", oErro)
END SEQUENCE

// --- Tratamento global ---
bOld := ErrorBlock({|oE| U_MEUTRADORB(oE)}) // instala . restaure com
ErrorBlock(bOld)
```

Propriedade do **oErro**

O que traz

:Description

O que deu errado

:ProcName / :ProcLine

Onde ocorreu (função / linha)

:Operation / :SubSystem

Operação e subsistema que falharam

Tabela	Papel	Chave	Campo virtual
SZ2	Mãe (Contatos)	Z2_FILIAL + Z2_CODIGO	Z2_NOME
SZ3	Filha (Interações)	Z3_FILIAL + Z3_CONTAT + Z3_SEQUEN	Z3_CODIGO , Z3_ASSUNT

🎯 Terminou o dicionário, a lib, as duas rotinas e a gravação segura? Você construiu, do zero e no seu tempo, um **sistema profissional completo**: CRUD relacionado, validações, integridade referencial e tratamento de erros. Dúvidas: [#duvidas-parte2-protheus](#) . 🔍